Wyniki etapu IV: Implementacja i testy

System wspomagania ZPI

Projektowanie oprogramowania

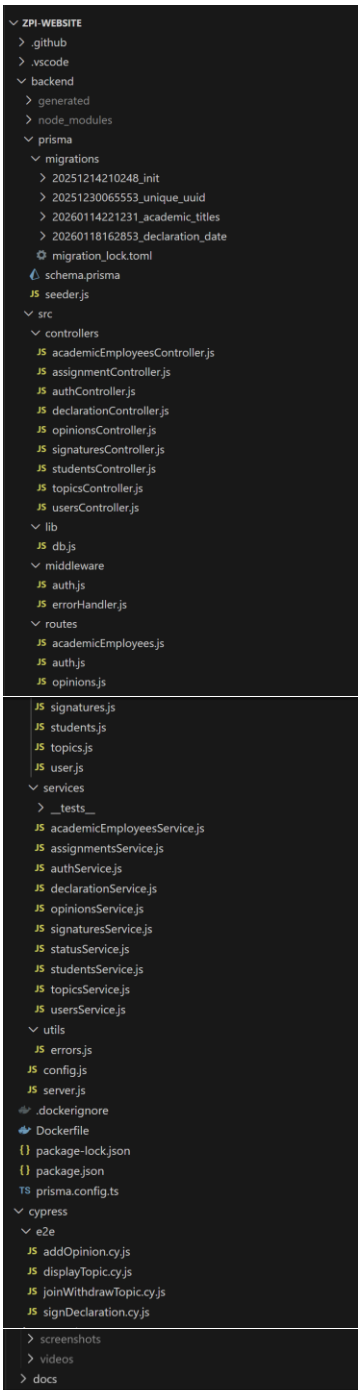
Skład zespołu:
Aleksandra Kozioł (280665),
Maja Kurczyna (280494)

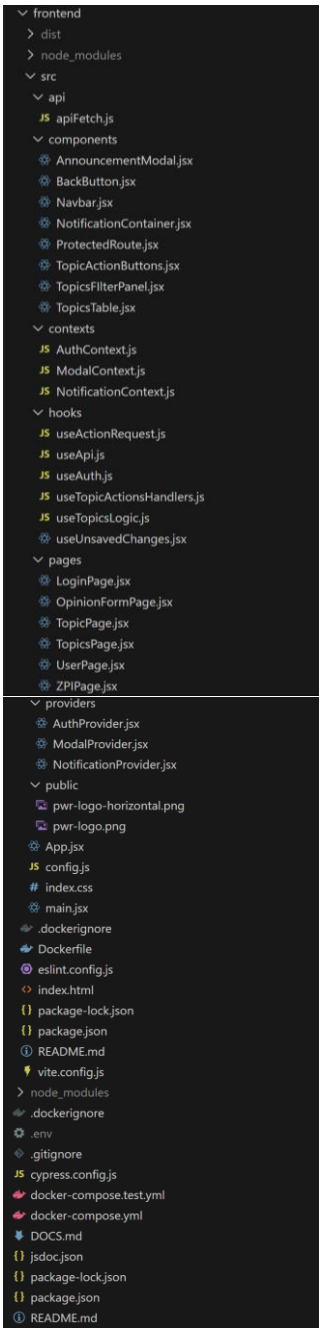
Prowadzący:
dr inż. Prof. Uczelni Bogumiła Hnatkowska

Implementacja

1. Struktura kodu

Link do repozytorium: *https://github.com/akoziolek/ZPI-Website*





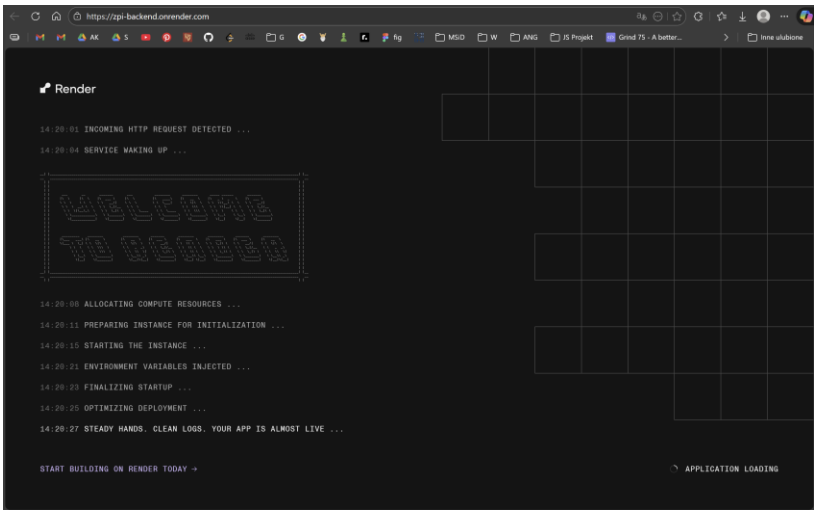
Strona jest dostępna online na platformie Render.

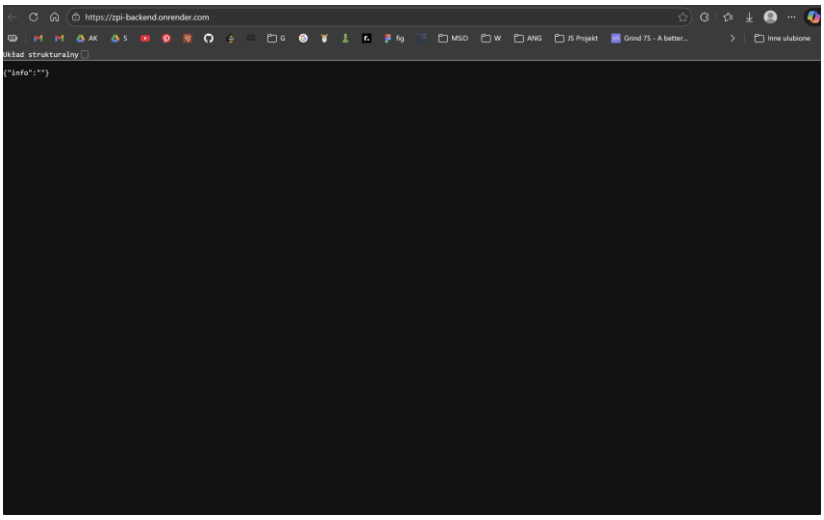
Wykorzystujemy darmowy plan, więc strony przechodzą w stan bezczynności po 15 minutach. Przy pierwszym uruchomieniu warto najpierw otworzyć adres backendu, aby zainicjować jego start i umożliwić poprawne pobieranie danych przez frontend.

Backend: https://zpi-backend.onrender.com/

Frontend: https://zpi-frontend.onrender.com/

Stan przed i po załadowaniem backendu





Do PU przeglądania tematów można zalogować się na dowolnego studenta. Dla PU dodawania opinii można zalogować się np. jako Adam Kot, Amelia Kot. Dla PU podpisywania deklaracji można zalogować się np. jako Piotr Zieliński, Katarzyna Wójcik. Dla PU dopisywania, a następnie wypisywanie z tematu można zalogować się np. jako Milena Polańska.

2. Generacja kodu

Pierwotna struktura projektu

```
kozio@Ola MINGW64 ~/OneDrive/Desktop/test
$ npm create vite@latest

> npx
> create-vite

◇  Project name:
│  vite-project
│
◇  Select a framework:
│  React
│
◇  Select a variant:
│  JavaScript
│
◇  Use rolldown-vite (Experimental)?:
│  No
│
◇  Install with npm and start now?
│  Yes
│
◇  Scaffolding project in C:\Users\kozio\OneDrive\Desktop\test\vite-project...
│
◇  Installing dependencies with npm...
```

```
> node_modules
v public
  vite.svg
v src
  > assets
  App.css
  App.jsx
  index.css
  main.jsx
.gitignore
eslint.config.js
index.html
package-lock.json
package.json
README.md
vite.config.js
```

Wykorzystałyśmy menadżer npm do wygenerowania szkieletu aplikacji, skonfigurował on zależności i je pobrał oraz stworzył skrypty budowania dzięki czemu mogłyśmy natychmiastowo przejść do rozwijania strony.

Zarządzanie bazą danych

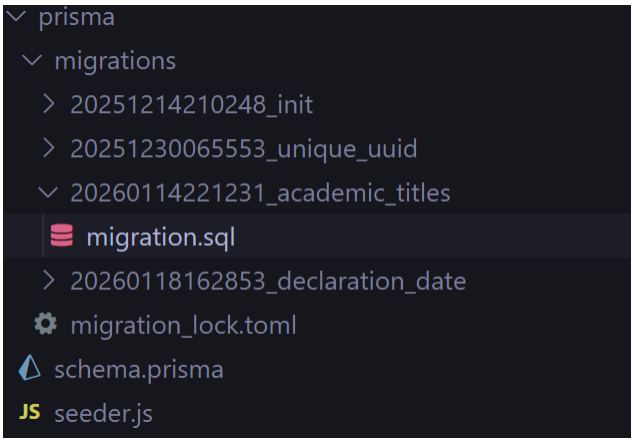
Dla każdej zmiany, która została wprowadzona w strukturze bazy danych od migracji inicjalizującej, została wygenerowana nowa migracja, do czego również została wykorzystana Prisma. Każda zmiana w pliku schematu, została automatycznie przerobiona na odpowiedni kod SQL. Przykładowo wprowadzono nową tabelę, przechowującą tytuły naukowe, a do tabeli zawierającej pracowników akademickich dodano odpowiednią nową kolumnę.

```
model AcademicTitle {
  id        Int                @id @default(autoincrement())
  full_name String             @unique()
  shortcut  String
  employees AcademicEmployee[]
}

model AcademicEmployee {
  user_id           Int            @id
  user              User           @relation(fields: [user_id], references: [user_id], onDelete: Cascade)
  academic_title_id Int?
  academic_title    AcademicTitle? @relation(fields: [academic_title_id], references: [id])

  opinions          Opinion[]
  topics            Topic[]
}
```

Zmiana związana z tytułami akademickimi znajduje się w folderze migracji.



Prisma odpowiadała za wygenerowanie SQL odpowiadającego wprowadzonym zmianom. W tym przypadku:

```sql
-- AlterTable
ALTER TABLE "AcademicEmployee" ADD COLUMN     "academic_title_id" INTEGER;

-- CreateTable
CREATE TABLE "AcademicTitle" (
    "id" SERIAL NOT NULL,
    "full_name" TEXT NOT NULL,
    "shortcut" TEXT NOT NULL,

    CONSTRAINT "AcademicTitle_pkey" PRIMARY KEY ("id")
);

-- CreateIndex
CREATE UNIQUE INDEX "AcademicTitle_full_name_key" ON "AcademicTitle"("full_name");

-- AddForeignKey
ALTER TABLE "AcademicEmployee" ADD CONSTRAINT "AcademicEmployee_academic_title_id_fkey" FOREIGN KEY ("academic_title_id") REFERENCES "AcademicTitle"("id") ON DELETE SET NULL ON UPDATE CASCADE;
```

Dokumentacja kodu

Przy użyciu narzędzia JSDoc mogłyśmy wygenerować stronę oraz plik .md z dokumentacją, opis znajduje się w punkcie 4.

3. Wzorce projektowe

Singleton

W projekcie wykorzystałyśmy wzorzec Singleton, aby zagwarantować istnienie wyłącznie jednej instancji klienta bazy danych *PrismaClient* w całym cyklu życia aplikacji. Decyzja ta jest podyktowana faktem, że tworzenie nowych instancji klientów przy każdym zapytaniu prowadziłoby do szybkiego wyczerpania dostępnych połączeń do bazy danych i spadku wydajności.
Nasza implementacja w JavaScripcie nie wykorzystuje klasycznego podejścia opartego na klasach lecz wykorzystuje mechanizm cache'owania modułów Node.js, w którym kod modułu wykonywany jest tylko raz, a jego eksporty są współdzielone pomiędzy kolejnymi importami. Dodatkowo zastosowano zmienną globalną, co pozwala zachować tę samą instancję klienta również w środowisku deweloperskim podczas Hot Reloadu frameworka, gdy moduły są przeładowywane, ale zmienne globalne nie są resetowane.

backend > src > lib > db.js > ...

```javascript
import { PrismaClient } from "../../generated/prisma/client.js";
import { PrismaPg } from "@prisma/adapter-pg";

// config to use only one prisma client/connection to the database

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL
});

const prismaClientSingleton = () => {
  return new PrismaClient({ adapter });
};

const prismaClient = global.prismaGlobal ?? prismaClientSingleton();

if (process.env.NODE_ENV !== "production") {
  global.prismaGlobal = prismaClient;
}

export default prismaClient;
```

DTO

Wzorzec projektowy powiązany z przesyłaniem danych pomiędzy warstwami aplikacji, aby odseparować wewnętrzny model danych (baza danych) od kontraktu zewnętrznego (odpowiedź API). Dzięki temu szczegóły implementacyjne są ukryte (np. klucz główne z bazy danych są zastąpione UUID), a struktura odpowiedzi jest spłaszczona przez co łatwiejsza do używania. DTO służy jedynie do transformacji danych, nie zawiera logiki biznesowej.
DTO w projekcie zaimplementowano jako obiekt literalny, ponieważ użycie klas w JavaScripcie nie zapewnia kontroli typów w czasie wykonywania i nie chroni przed przekazaniem danych o nieprawidłowej strukturze, więc w naszym przypadku ich użycie nie przyniosłoby dodatkowych korzyści.

backend > src > services > topicsService.js > ...

```javascript
export function mapTopicToDto(topic) {
    return {
        uuid: topic.uuid,
         name: topic.name,
         description: topic.description,
         status_name: topic.status.status_name,

         supervisor: topic.employee ? {
             uuid: topic.employee.user.uuid,
             name: topic.employee.user.name,
             surname: topic.employee.user.surname,
             full_academic_title: topic.employee.academic_title?.full_name,
             shortcut_academic_title: topic.employee.academic_title?.shortcut,
         } : null,

         students: topic.students.map(student => ({
             uuid: student.user.uuid,
             index: student.index,
             name: student.user.name,
             surname: student.user.surname,
         })),

         opinion: topic.opinion ? {
             argumentation: topic.opinion.argumentation,
             author: {
                 uuid: topic.opinion.employee.user.uuid,
                 name: topic.opinion.employee.user.name,
                 surname: topic.opinion.employee.user.surname
             },
         } : null
    };
}
```

```
54  export const getAllTopics = async (search) => {
55      const topics = await prismaClient.topic.findMany({
56          where: search ? {
57              name: { contains: search, mode: 'insensitive' }
58          } : {},
59          include: {
60              status: true,
61              employee: { include: { user: true, academic_title: true } },
62              students: { include: { user: true } }
63          }
64      });
65      return topics.map(mapTopicToDto);
66  };
```

Transaction script

Wzorzec ten polega na organizowaniu logiki biznesowej w formie proceduralnej, gdzie pojedyncza procedura obsługuje kompleksowo konkretne żądanie od początku do końca. W naszym przypadku są to usługi świadczone przez serwisy backendu tj. podpisywanie deklaracji, z których korzystają kontrolery. Funkcja odpowiedzialna jest za pobranie danych, przeprowadzenie serii walidacji, wykonanie głównej akcji oraz zmianę stanu całego obiektu. Podejście to zapewniło, że logika biznesowa nie jest rozrzucona na kilka klas, a trzymana w jednym miejscu, przez co łatwo było ją modyfikować.

```
backend > src > services > JS declarationService.js > ...
31  export async function signDeclaration(topicUuid, userId) {
32      const topic = await prismaClient.topic.findUnique({
33          where: { uuid: topicUuid },
34          include: {
35              status: true,
36              students: true,
37              declaration: { include: { signatures: true }, },
38          },
39      });
40
41      if (!topic) {
42          throw new NotFoundError("Topic not found");
43      }
44
45      if (!topic.declaration) {
46          throw new NotFoundError("Declaration not found")
47      }
48
49      if (topic.status.status_name !== STATUSES.PREPARING) {
50          throw new ValidationError("Topic must be in 'W przygotowaniu do słożenia wniosku' status to be signed");
51      }
52
53      const assignedStudent = topic.students.some((student) => student.user_id === userId)
54
55      if (!assignedStudent) {
56          throw new ValidationError("Student must be assigned to topic to sign it");
57      }
58
59      const declarationId = topic.declaration.declaration_id
60
61      const signatureExists = topic.declaration.signatures.some((signature) => signature.user_id === userId);
62
63      if (signatureExists) {
64          throw new ValidationError("ALREADY_SIGNED");
65      }
66
67      await createSignature(userId, declarationId);
68
69      const studentCount = topic.students.length
70      const signatureCount = topic.declaration.signatures.length
71
72      if (studentCount === (signatureCount+1)) {
73          const submittedStatus = await findStatus(STATUSES.SUBMITTED);
74          if (submittedStatus) {
75              await updateStatus(topicUuid, submittedStatus.status_id);
76          }
77      }
78  }
```

4. Dokumentacja kodu

Dokumentacja kodu została wygenerowana przy pomocy JSDoc. Najważniejsze funkcje, tj. dla eksportowanych funkcji kontrolerów, serwisów oraz dla hooków zostały stworzone opisujące komentarze. Przykładowy komentarz dla funkcji *joinTopic* z *assignmentService.js*:

```
/**
 * Join a topic by its UUID for a given user.
 *
 * Validates that the topic exists, is in an open status, and that the
 * maximum student capacity hasn't been reached. On success the student's
 * record is updated to reference the topic.
 *
 * @param {string} topicUuid - The UUID of the topic to join.
 * @param {number} userId - The internal database id of the user (student).
 * @throws {NotFoundError} If the topic does not exist.
 * @throws {ValidationError} If the topic status is not open or the student limit is reached.
 * @returns {Promise<void>} Resolves when the join operation completes.
 */
export async function joinTopic(topicUuid, userId) {
```

Używając narzędzia JSDoc możliwe było wygenerowanie dokumentacji, którą można przeglądać w formie strony (wygenerowane pliki znajdują się w folderze *docs*). Dodatkowo wygenerowano dokumentację zapisaną w formacie markdown – plik *DOCS.md*.

Przykładowy fragment dokumentacji w formacie html dla *joinTopic*:

JSDoc: Global

localhost:8080/global.html#joinTopic

joinTopic(topicUuid, userId) → {Promise.<void>}

Join a topic by its UUID for a given user. Validates that the topic exists, is in an open status, and that the maximum student capacity hasn't been reached. On success the student's record is updated to reference the topic.

Parameters:

Name	Type	Description
topicUuid	string	The UUID of the topic to join.
userId	number	The internal database id of the user (student).

Source: backend/src/services/assignmentsService.js, line 38

Throws:

- If the topic does not exist.
  Type
  NotFoundError
- If the topic status is not open or the student limit is reached.
  Type
  ValidationError

Returns:

Resolves when the join operation completes.

Type
Promise.<void>

Przykładowy fragment dokumentacji w formacie markdown dla *joinTopic*:

```
joinTopic(topicUuid, userId) ⇒ Promise.<void>
    Join a topic by its UUID for a given user.

    Validates that the topic exists, is in an open status, and that the maximum student capacity hasn't been reached. On success the student's record is updated to reference the topic.
```

Testowanie

1. Testy systemowe – przypadki/procedury testowe

1.1 Podpisanie deklaracji [Maja Kurczyna]

TC ID: 001
Idea testowa: Udane podpisanie deklaracji
Warunki wstępne:
W systemie istnieje temat ze statusem *W przygotowaniu do złożenia wniosku*:
Tabela 1. Szczegóły tematu

L.p.	Nazwa	Opis	Status	Opiekun	Studenci	Podpisy studentów
1	Architektura mikroserwisów w chmurze	Projekt i implementacja skalowalnego systemu opartego na mikroserwisach z wykorzystaniem technologii Docker i Kubernetes w środowisku chmury obliczeniowej.	W przygotowaniu do złożenia wniosku	Marcin Nowicki	Anna Nowak, Piotr Zieliński, Katarzyna Wójcik	-

W systemie istnieje student dopisany do tego tematu, który jeszcze nie został przez niego podpisany:
Tabela 2. Dane studenta

L.p.	Imię	Nazwisko	E-mail
1	Anna	Nowak	anna.nowak.234567@pwr.edu.pl
2	Piotr	Zieliński	piotr.zieliński.345678@pwr.edu.pl

Powyższy student nie jest ostatnią osobą, która musi podpisać deklarację. Powyższy użytkownik jest zalogowany na swoje konto.

Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję *Wyświetl* przy temacie, do którego jest dopisany.		System wyświetla szczegóły tematu (patrz tabela 1.).
Użytkownik wybiera opcję *Podpisz*.		System wyświetla komunikat *Podpisano deklarację!*.
Użytkownik wybiera opcję *Zamknij*.		System przechodzi na stronę wyświetlania

		szczegółów tematu.

TC ID: 002
Idea testowa: Podpisanie deklaracji – ostatni podpisujący
Warunki wstępne:
W systemie istnieje temat ze statusem *W przygotowaniu do złożenia wniosku*:
Tabela 1. Szczegóły tematu

L.p.	Nazwa	Opis	Status	Opiekun	Studenci	Podpisy studentów
1	Architektura mikroserwisów w chmurze	Projekt i implementacja skalowalnego systemu opartego na mikroserwisach z wykorzystaniem technologii Docker i Kubernetes w środowisku chmury obliczeniowej.	W przygotowaniu do złożenia wniosku	Marcin Nowicki	Anna Nowak, Piotr Zieliński, Katarzyna Wójcik	Anna Nowak, Piotr Zieliński

W systemie istnieje student dopisany do tego tematu, który jeszcze nie został przez niego podpisany:
Tabela 2. Dane studenta

L.p.	Imię	Nazwisko	E-mail
1	Katarzyna	Wójcik	katarzyna.wójcik.456789@pwr.edu.pl

Powyższy student jest ostatnią osobą, która musi podpisać deklarację. Powyższy użytkownik jest zalogowany na swoje konto.

Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję *Wyświetl* przy temacie, do którego jest dopisany.		System wyświetla szczegóły tematu (patrz tabela 1.).
Użytkownik wybiera opcję *Podpisz*.		System wyświetla komunikat *Podpisano deklarację!*. Status tematu został zmieniony na *Złożony*.
Użytkownik wybiera opcję *Zamknij*.		System przechodzi na stronę wyświetlania szczegółów tematu.

TC ID: 003
Idea testowa: Podpisanie deklaracji – już podpisano
Warunki wstępne:
W systemie istnieje temat ze statusem *W przygotowaniu do złożenia wniosku*:
Tabela 1. Szczegóły tematu

L.p.	Nazwa	Opis	Status	Opiekun	Studenci	Podpisy studentów
1	Architektura	Projekt i	W	Marcin	Anna Nowak,	Katarzyna

	mikroserwisów w chmurze	implementacja skalowalnego systemu opartego na mikroserwisach z wykorzystaniem technologii Docker i Kubernetes w środowisku chmury obliczeniowej.	przygotowaniu do złożenia wniosku	Nowicki	Piotr Zieliński, Katarzyna Wójcik	Wójcik

W systemie istnieje student dopisany do tego tematu, który już został przez niego podpisany:
Tabela 2. Dane studenta

L.p.	Imię	Nazwisko	E-mail
1	Katarzyna	Wójcik	katarzyna.wójcik.456789@pwr.edu.pl

Powyższy użytkownik jest zalogowany na swoje konto.

Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję *Wyświetl* przy temacie, do którego jest dopisany.		System wyświetla szczegóły tematu (patrz tabela 1.). Opcja *Podpisz* jest niedostępna.

TC ID: 004
Idea testowa: Niepoprawne podpisanie deklaracji – student niedopisany do tematu
Warunki wstępne:
W systemie istnieje temat ze statusem *W przygotowaniu do złożenia wniosku*:
Tabela 1. Szczegóły tematu

L.p.	Nazwa	Opis	Status	Opiekun	Studenci	Podpisy studentów
1	Architektura mikroserwisów w chmurze	Projekt i implementacja skalowalnego systemu opartego na mikroserwisach z wykorzystaniem technologii Docker i Kubernetes w środowisku chmury obliczeniowej.	W przygotowaniu do złożenia wniosku	Marcin Nowicki	Anna Nowak, Piotr Zieliński, Katarzyna Wójcik	Katarzyna Wójcik

W systemie istnieje student niedopisany do tego tematu:
Tabela 2. Dane studenta

L.p.	Imię	Nazwisko	E-mail

1	Kamil	Tomczak	kamil.tomczak.412345@pwr.edu.pl

Powyższy użytkownik jest zalogowany na swoje konto.

Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję *Wyświetl* przy temacie *W przygotowaniu do złożenia wniosku*.		System wyświetla szczegóły tematu (patrz tabela 1.). Opcja *Podpisz* jest niedostępna.

1.2 Dopisanie do tematu [Maja Kurczyna]

TC ID: 001
Idea testowa: Poprawne dopisanie do tematu
Warunki wstępne:
W systemie istnieje co najmniej jeden temat o statusie *Otwarty,* w którym nie osiągnięto limitu studentów:
Tabela 1. Szczegóły tematu

L.p.	Nazwa	Opis	Status	Opiekun	Studenci	Podpisy studentów
1	Wpływ AI na procesy rekrutacyjne	Analiza wykorzystania algorytmów uczenia maszynowego w automatyzacji selekcji CV oraz ocenie kompetencji miękkich kandydatów podczas rozmów rekrutacyjnych.	Otwarty	-	-	-

W systemie istnieje student, który nie jest dopisany do żadnego tematu:
Tabela 2. Dane studenta

L.p.	Imię	Nazwisko	E-mail
1	Milena	Polańska	milena.polańska.589014@pwr.edu.pl
2	Wiktoria	Kaczor	wiktoria.kaczor.601236@pwr.edu.pl
3	Artur	Zawadzki	artur.zawadzki.612347@pwr.edu.pl
4	Hubert	Szulc	hubert.szulc.590125@pwr.edu.pl
5	Kacper	Kalinowski	kacper.kalinowski.578903@pwr.edu.pl

Powyższy użytkownik jest zalogowany na swoje konto.

Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję *Wyświetl* przy temacie, który ma status *Otwarty* oraz liczbę studentów <5.		System wyświetla szczegóły tematu (patrz tabela 1.).
Użytkownik wybiera opcję *Zapisz się*.		System wyświetla komunikat *Dopisano do tematu!*.

Użytkownik wybiera opcję *Zamknij*.		System przechodzi na stronę wyświetlania szczegółów tematu.

TC ID: 002
Idea testowa: Dopisanie do tematu – limit miejsc osiągnięty
Warunki wstępne:
W systemie istnieje co najmniej jeden temat o statusie *Otwarty,* w którym osiągnięto limit studentów:
Tabela 1. Szczegóły tematu

L.p.	Nazwa	Opis	Status	Opiekun	Studenci	Podpisy studentów
1	Wpływ AI na procesy rekrutacyjne	Analiza wykorzystania algorytmów uczenia maszynowego w automatyzacji selekcji CV oraz ocenie kompetencji miękkich kandydatów podczas rozmów rekrutacyjnych.	Otwarty	-	Milena Polańska, Wiktoria Kaczor, Artur Zawadzki, Hubert Szulc, Kacper Kalinowski	-

W systemie istnieje student, który nie jest dopisany do żadnego tematu:
Tabela 2. Dane studenta

L.p.	Imię	Nazwisko	E-mail
1	Zuzanna	Błaszczyk	zuzanna.błaszczyk.567892@pwr.edu.pl

Powyższy użytkownik jest zalogowany na swoje konto.

Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję *Wyświetl* przy temacie, który ma status *Otwarty* oraz liczbę studentów ==5.		System wyświetla szczegóły tematu (patrz tabela 1.).
Użytkownik wybiera opcję *Zapisz się*.		System wyświetla komunikat *Brak miejsc!*.
Użytkownik wybiera opcję *Zamknij*.		System przechodzi na stronę wyświetlania szczegółów tematu.

TC ID: 003
Idea testowa: Dopisanie do tematu – już dopisano
Warunki wstępne:
W systemie istnieje co najmniej jeden temat o statusie *Otwarty,* w którym nie osiągnięto limit studentów:
Tabela 1. Szczegóły tematu

L.p.	Nazwa	Opis	Status	Opiekun	Studenci	Podpisy studentów

1	Wpływ AI na procesy rekrutacyjne	Analiza wykorzystania algorytmów uczenia maszynowego w automatyzacji selekcji CV oraz ocenie kompetencji miękkich kandydatów podczas rozmów rekrutacyjnych.	Otwarty	-	Zuzanna Błaszczyk	-

W systemie istnieje student, który nie jest dopisany do żadnego tematu:
Tabela 2. Dane studenta

L.p.	Imię	Nazwisko	E-mail
1	Zuzanna	Błaszczyk	zuzanna.błaszczyk.567892@pwr.edu.pl

Powyższy użytkownik jest zalogowany na swoje konto.

Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję *Wyświetl* przy temacie, który ma status *Otwarty* oraz liczbę studentów <5.		System wyświetla szczegóły tematu (patrz tabela 1.). Opcja *Zapisz się* jest niedostępna.

1.3 Wypisanie się z tematu [Maja Kurczyna]

TC ID: 001
Idea testowa: Poprawne wypisanie się z tematu
Warunki wstępne:
W systemie istnieje co najmniej jeden temat o statusie *Otwarty,* w którym znajdują się dopisani studenci:
Tabela 1. Szczegóły tematu

L.p.	Nazwa	Opis	Status	Opiekun	Studenci	Podpisy studentów
1	Wpływ AI na procesy rekrutacyjne	Analiza wykorzystania algorytmów uczenia maszynowego w automatyzacji selekcji CV oraz ocenie kompetencji miękkich kandydatów podczas rozmów rekrutacyjnych.	Otwarty	-	Milena Polańska, Wiktoria Kaczor, Artur Zawadzki, Hubert Szulc, Kacper Kalinowski	-

W systemie istnieje student, który jest dopisany do tematu:
Tabela 2. Dane studenta

L.p.	Imię	Nazwisko	E-mail
1	Milena	Polańska	milena.polańska.589014@pwr.edu.pl
2	Wiktoria	Kaczor	wiktoria.kaczor.601236@pwr.edu.pl
3	Artur	Zawadzki	artur.zawadzki.612347@pwr.edu.pl
4	Hubert	Szulc	hubert.szulc.590125@pwr.edu.pl
5	Kacper	Kalinowski	kacper.kalinowski.578903@pwr.edu.pl

Powyższy użytkownik jest zalogowany na swoje konto.

Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję *Wyświetl* przy temacie, do którego jest dopisany.		System wyświetla szczegóły tematu (patrz tabela 1.).
Użytkownik wybiera opcję *Wypisz się*.		System wyświetla komunikat *Wypisano się z tematu!*.
Użytkownik wybiera opcję *Zamknij*.		System przechodzi na stronę wyświetlania szczegółów tematu.

TC ID: 002
Idea testowa: Niepoprawne wypisanie się z tematu - student nie jest dopisany
Warunki wstępne:
W systemie istnieje co najmniej jeden temat o statusie *Otwarty*:
Tabela 1. Szczegóły tematu

L.p.	Nazwa	Opis	Status	Opiekun	Studenci	Podpisy studentów
1	Wpływ AI na procesy rekrutacyjne	Analiza wykorzystania algorytmów uczenia maszynowego w automatyzacji selekcji CV oraz ocenie kompetencji miękkich kandydatów podczas rozmów rekrutacyjnych.	Otwarty	-	-	-

W systemie istnieje student, który nie jest dopisany do tematu:
Tabela 2. Dane studenta

L.p.	Imię	Nazwisko	E-mail
1	Milena	Polańska	milena.polańska.589014@pwr.edu.pl

Powyższy użytkownik jest zalogowany na swoje konto.

Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję *Wyświetl* przy temacie o statusie *Otwarty*.		System wyświetla szczegóły tematu (patrz tabela 1.). Opcja *Wypisz się* jest niedostępna.

1.4 Wyświetlanie tematów i szczegółów przez Studenta [Aleksandra Kozioł]

Tematy, studenci, prowadzący dostępne w systemie [DT1] – warunek wstępny.

Studenci

Lp.	Imię	Nazwisko	Mail
1	Artur	Zawadzki	artur.zawadzki.612347@pwr.edu.pl
2	Patryk	Witkowski	patryk.witkowski.534569@pwr.edu.pl
3	Iga	Piasecka	iga.piasecka.489014@pwr.edu.pl
4	Norbert	Kołodziejski	norbert.kołodziej.467892@pwr.edu.pl
5	Anna	Nowak	anna.nowak.234567@pwr.edu.pl
6	Piotr	Zieliński	piotr.zieliński.345678@pwr.edu.pl
7	Katarzyna	Wójcik	Katarzyna.wójcik.4567@pwr.edu.pl
8	Krzysztof	Kowalczyk	krzysztof.kowalczyk.567890@pwr.edu.pl
9	Magdalena	Lewandowska	magdalena.lewandowska.678901@pwr.edu.pl
10	Tomasz	Kaczmarek	tomasz.kaczmarek.789012@pwr.edu.pl
11	Natalia	Mazur	natalia.mazur.890123@pwr.edu.pl
12	Patrycja	Kubiak	patrycja.kubiak.345688@pwr.edu.pl
13	Adrian	Bąk	adrian.bąk.356789@pwr.edu.pl
14	Julia	Ostrowska	julia.ostrowska.367890@pwr.edu.pl
15	Szymon	Urbański	szymon.urbański.378901@pwr.edu.pl
16	Paulina	Cieślak	paulina.cieślak.389012@pwr.edu.pl
17	Damian	Kruk	damian.kruk.390123@pwr.edu.pl
18	Sandra	Bednarek	sandra.bednarek.401234@pwr.edu.pl
19	Kamil	Tomczak	kamil.tomczak.412345@pwr.edu.pl

Prowadzący

Lp.	Tytuł	Imię	Nazwisko	Mail
1	-	Piotr	Jastrzębski	piotr.jastrzębski.8@pwr.edu.pl
2	dr inż.	Kuba	Jarzębski	kuba.jarzębski.12@pwr.edu.pl
3	prof. dr hab.	Marcin	Nowicki	marcin.nowicki.1@pwr.edu.pl
4	mgr	Alicja	Kozłowska	alicja.kozłowska.2@pwr.edu.pl
5	-	Radosław	Sowa	radosław.sowa.10@pwr.edu.pl
6	-	Maciej	Łódź	maciej.łódź.11@pwr.edu.pl

Tematy

Lp.	**Nazwa**	**Opis**	**Status**	**Opiekun (tytuł + imię +**	**Studenci (imię + nazwisko)**	**Podpisy**

				nazwisko)		
1	Wpływ AI na procesy rekrutacyjne	Analiza wykorzystania algorytmów uczenia maszynowego w automatyzacji selekcji CV oraz ocenie kompetencji miękkich kandydatów podczas rozmów rekrutacyjnych.	Otwarty	-	-	-
2	Aplikacja mobilna wspierająca naukę	Stworzenie aplikacji mobilnej umożliwiającej planowanie nauki, śledzenie postępów oraz powtórki materiału.	Otwarty	Piotr Jastrzębski	Artur Zawadzki	-
3	Bezpieczeństwo aplikacji webowych	Analiza najczęstszych podatności aplikacji webowych (OWASP Top 10) oraz implementacja mechanizmów zabezpieczających.	W przygotowaniu do złożenia	dr inż. Kuba Jarzębski	Patryk Witkowski Iga Piasecka, Norbert Kołodziejski	dr inż. Kuba Jarzębski
4	Architektura mikroserwisów w chmurze	Projekt i implementacja skalowalnego systemu opartego na mikroserwisach z wykorzystaniem technologii Docker i Kubernetes w środowisku chmury obliczeniowej.	W przygotowaniu do złożenia	prof. dr hab. Marcin Nowicki	Anna Nowak, Piotr Zieliński, Katarzyna Wójcik	prof. dr hab. Marcin Nowicki, Anna Nowak, Piotr Zieliński
5	Modelowanie danych w NoSQL	Badanie wydajności oraz struktur przechowywania danych w bazach dokumentowych i grafowych w porównaniu do tradycyjnych relacyjnych baz danych.	Złożony	mgr Alicja Kozłowska	Krzysztof Kowalczyk, Magdalena Lewandowska Tomasz Kaczmarek, Natalia Mazur	mgr Alicja Kozłowska, Krzysztof Kowalczyk, Magdalena Lewandowska, Tomasz Kaczmarek, Natalia Mazur
6	Machine	Opracowanie	Zatwierdzony	Radosław	Patrycja	Radosław

	Learning dla detekcji oszustw	modelu klasyfikacyjnego wykrywającego anomalie w transakcjach finansowych w czasie rzeczywistym, mającego na celu zapobieganie nadużyciom kart płatniczych.		Sowa	Kubiak, Adrian Bąk, Julia Ostrowska, Szymon Urbański, Paulina Cieślak	Sowa, Patrycja Kubiak, Adrian Bąk, Julia Ostrowska, Szymon Urbański, Paulina Cieślak
7	Zastosowanie blockchain w logistyce	Wykorzystanie technologii rozproszonych rejestrów (DLT) do zwiększenia przejrzystości łańcucha dostaw oraz automatyzacji płatności poprzez inteligentne kontrakty (Smart Contracts).	Odrzucony	Maciej Łódź	Damian Kruk, Sandra Bednarek, Kamil Tomczak	Maciej Łódź, Damian Kruk, Sandra Bednarek, Kamil Tomczak

TC ID: 001 Idea testowa: Wyświetlenie komunikat o braku danych, gdy lista tematów jest pusta Warunki wstępne: W systemie nie ma żadnych tematów. Użytkownik jest zalogowanym studentem.		
Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję wyświetlenia listy tematów.		System wyświetla stronę z listą tematów z komunikatem „Brak tematów do wyświetlenia”.

TC ID: 002 Idea testowa: Wyświetlanie listy tematów Warunki wstępne: System zawiera tematy zdefiniowane w DT1. Użytkownik jest zalogowanym studentem.		
Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję wyświetlenia listy tematów.		System wyświetla stronę z listą tematów (zgodnie z danymi z tabeli DT1) zawierających: 1. Nazwę 2. Status 3. Opiekuna (wyświetlonego jako połączony ciąg: [Tytuł] [Imię] [Nazwisko] lub jeżeli nie istnieje tekst *Brak opiekuna*) 4. Liczbę osób w zespole (wartość cyfrowa zgodna z liczbą osób przypisanych do tematu) 5. Przycisk z tekstem „Wyświetl”.

TC ID: 003 Idea testowa: Wyświetlanie szczegółów tematu Otwartego z perspektywy studenta nie w zespole Warunki wstępne: W systemie znajduje się przynajmniej jeden temat, który posiada status Otwarty (zgodnie z DT1). Użytkownik jest zalogowanym studentem, który nie należy do tego zespołu.		
Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję Wyświetl tematu o statusie Otwarty z listy tematów.		System wyświetla stronę szczegółów tematu (zgodnie z danymi z tabeli DT1), na której widoczne są: 1. Nazwa 2. Status (Otwarty) 3. Opiekun (ciąg: [Tytuł] [Imię] [Nazwisko] lub znak „-” w przypadku braku). 4. Sekcja „Skład zespołu” ze studentami przypisanymi do tematu (każdy w osobnym wierszy: [Imię] [Nazwisko] lub znak „-” w przypadku braku) 5. Opis lub w przypadku braku „-”. 6. Przycisk z tekstem „Zapisz się”.

TC ID: 004 Idea testowa: Wyświetlanie szczegółów tematu Otwartego z perspektywy studenta w zespole Warunki wstępne: W systemie znajduje się przynajmniej jeden temat, który posiada status Otwarty (zgodnie z DT1). Użytkownik jest zalogowanym studentem, który należy do tego zespołu.		
Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję Wyświetl tematu o statusie Otwarty z listy tematów.		System wyświetla stronę szczegółów tematu (zgodnie z danymi z tabeli DT1), na której widoczne są: 1. Nazwa 2. Status (Otwarty) 3. Opiekun (ciąg: [Tytuł] [Imię] [Nazwisko] lub znak „-” w przypadku braku). 4. Sekcja „Skład zespołu” ze studentami przypisanymi do tematu (każdy w osobnym wierszy: [Imię] [Nazwisko] lub znak „-” w przypadku braku) 5. Opis lub w przypadku braku „-”. 6. Przycisk z tekstem „Wypisz się”.

TC ID: 005 Idea testowa: Wyświetlanie szczegółów tematu W przygotowaniu do złożenia z perspektywy studenta nie w zespole Warunki wstępne: W systemie znajduje się przynajmniej jeden temat, który posiada status W przygotowaniu do

złożenia (zgodnie z DT1). Użytkownik jest zalogowanym studentem, który nie należy do tego zespołu.		
Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję *Wyświetl* tematu o statusie W przygotowaniu do złożenia z listy tematów.		System wyświetla stronę szczegółów tematu (zgodnie z danymi z tabeli DT1), na której widoczne są: 1. Nazwa 2. Status (W przygotowaniu do złożenia) 3. Opiekun (ciąg: [Tytuł] [Imię] [Nazwisko] lub znak „-” w przypadku braku). 4. Sekcja „Skład zespołu” ze studentami przypisanymi do tematu (każdy w osobnym wierszy: [Imię] [Nazwisko] lub znak „-” w przypadku braku) 5. Sekcja „Podpisano przez:” – wykaz osób, które złożyły podpis: • Opiekun: wyświetlany jako ciąg: [Tytuł] [Imię] [Nazwisko]. • Studenci: wyświetlani jako ciąg: [Imię] [Nazwisko]. • W przypadku braku podpisów wyświetlany jest znak „-”. 6. Opis lub w przypadku braku „-”.

TC ID: 006 Idea testowa: Wyświetlanie szczegółów tematu W przygotowaniu do złożenia z perspektywy studenta w zespole, który nie podpisał deklaracji Warunki wstępne: W systemie znajduje się przynajmniej jeden temat, który posiada status W przygotowaniu do złożenia (zgodnie z DT1). Użytkownik jest zalogowanym studentem, który należy do tego zespołu i nie dokonał podpisu deklaracji tego tematu.		
Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję Wyświetl tematu o statusie W przygotowaniu do złożenia z listy tematów.		System wyświetla stronę szczegółów tematu (zgodnie z danymi z tabeli DT1), na której widoczne są: 1. Nazwa 2. Status (W przygotowaniu do złożenia) 3. Opiekun (ciąg: [Tytuł] [Imię] [Nazwisko] lub znak „-” w przypadku braku). 4. Sekcja „Skład zespołu” ze studentami przypisanymi do tematu (każdy w osobnym

		wierszy: [Imię] [Nazwisko] lub znak „-” w przypadku braku) 5. Sekcja „Podpisano przez:” – wykaz osób, które złożyły podpis: • Opiekun: wyświetlany jako ciąg: [Tytuł] [Imię] [Nazwisko]. • Studenci: wyświetlani jako ciąg: [Imię] [Nazwisko]. • W przypadku braku podpisów wyświetlany jest znak „-”. 6. Opis lub w przypadku braku „-”. 7. Przycisk z tekstem „Podpisz”.

TC ID: 007 Idea testowa: Wyświetlanie szczegółów tematu W przygotowaniu do złożenia z perspektywy studenta w zespole, który podpisał deklaracje Warunki wstępne: W systemie znajduje się przynajmniej jeden temat, który posiada status W przygotowaniu do złożenia (zgodnie z DT1). Użytkownik jest zalogowanym studentem, który należy do tego zespołu i dokonał już podpisu deklaracji tego tematu.		
Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję *Wyświetl* tematu o statusie W przygotowaniu do złożenia z listy tematów.		System wyświetla stronę szczegółów tematu (zgodnie z danymi z tabeli DT1), na której widoczne są: 1. Nazwa 2. Status (W przygotowaniu do złożenia) 3. Opiekun (ciąg: [Tytuł] [Imię] [Nazwisko] lub znak „-” w przypadku braku). 4. Sekcja „Skład zespołu” ze studentami przypisanymi do tematu (każdy w osobnym wierszy: [Imię] [Nazwisko] lub znak „-” w przypadku braku) 5. Sekcja „Podpisano przez:” – wykaz osób, które złożyły podpis: • Opiekun: wyświetlany jako ciąg: [Tytuł] [Imię] [Nazwisko]. • Studenci: wyświetlani jako ciąg: [Imię] [Nazwisko]. • W przypadku braku podpisów w danej kategorii wyświetlany jest znak „-”.

		6. Opis lub w przypadku braku „-”.

TC ID: 008 Idea testowa: Wyświetlanie szczegółów tematu Złożonego Warunki wstępne: W systemie znajduje się przynajmniej jeden temat, który posiada status *Złożony* (zgodnie z DT1). Użytkownik jest zalogowanym studentem.		
Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję Wyświetl tematu o statusie Złożony z listy tematów.		System wyświetla stronę szczegółów tematu (zgodnie z danymi z tabeli DT1), na której widoczne są: 1. Nazwa 2. Status (W przygotowaniu do złożenia) 3. Opiekun (ciąg: [Tytuł] [Imię] [Nazwisko] lub znak „-” w przypadku braku). 4. Sekcja „Skład zespołu” ze studentami przypisanymi do tematu (każdy w osobnym wierszy: [Imię] [Nazwisko] lub znak „-” w przypadku braku) 5. Opis lub w przypadku braku „-”.

TC ID: 009 Idea testowa: Wyświetlanie szczegółów tematu Zaakceptowanego z perspektywy studenta nie w zespole Warunki wstępne: W systemie znajduje się przynajmniej jeden temat, który posiada status Zaakceptowany (zgodnie z DT1). Użytkownik jest zalogowanym studentem, który nie jest w zespole.		
Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję Wyświetl tematu o statusie Zaakceptowany z listy tematów.		System wyświetla stronę szczegółów tematu (zgodnie z danymi z tabeli DT1), na której widoczne są: 1. Nazwa 2. Status (Zaakceptowany) 3. Opiekun (ciąg: [Tytuł] [Imię] [Nazwisko] lub znak „-” w przypadku braku). 4. Sekcja „Skład zespołu” ze studentami przypisanymi do tematu (każdy w osobnym wierszy: [Imię] [Nazwisko] lub znak „-” w przypadku braku) 5. Opis lub w przypadku braku „-”.

TC ID: 010 Idea testowa: Wyświetlanie szczegółów tematu Zaakceptowanego z perspektywy studenta w zespole Warunki wstępne: W systemie znajduje się przynajmniej jeden temat, który posiada status Zaakceptowany

(zgodnie z DT1). Użytkownik jest zalogowanym studentem, który jest w zespole.		
Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję *Wyświetl* tematu o statusie Zaakceptowany z listy tematów.		System wyświetla stronę szczegółów tematu (zgodnie z danymi z tabeli DT1), na której widoczne są: 1. Nazwa 2. Status (Zaakceptowany) 3. Opiekun (ciąg: [Tytuł] [Imię] [Nazwisko] lub znak „-” w przypadku braku). 4. Sekcja „Skład zespołu” ze studentami przypisanymi do tematu (każdy w osobnym wierszy: [Imię] [Nazwisko] lub znak „-” w przypadku braku) 5. Opis lub w przypadku braku „-”. 6. Przycisk z tekstem „Zobacz Opinię”.

TC ID: 011 Idea testowa: Wyświetlanie szczegółów tematu Odrzuconego z perspektywy studenta nie w zespole Warunki wstępne: W systemie znajduje się przynajmniej jeden temat, który posiada status Odrzucony (zgodnie z DT1). Użytkownik jest zalogowanym studentem, który nie jest w zespole.		
Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję *Wyświetl* tematu o statusie Odrzucony z listy tematów.		System wyświetla stronę szczegółów tematu (zgodnie z danymi z tabeli DT1), na której widoczne są: 1. Nazwa 2. Status (Odrzucony) 3. Opiekun (ciąg: [Tytuł] [Imię] [Nazwisko] lub znak „-” w przypadku braku). 4. Sekcja „Skład zespołu” ze studentami przypisanymi do tematu (każdy w osobnym wierszy: [Imię] [Nazwisko] lub znak „-” w przypadku braku) 5. Opis lub w przypadku braku „-”.

TC ID: 012 Idea testowa: Wyświetlanie szczegółów tematu Odrzuconego z perspektywy studenta w zespole Warunki wstępne: W systemie znajduje się przynajmniej jeden temat, który posiada status Odrzucony (zgodnie z DT1). Użytkownik jest zalogowanym studentem, który jest w zespole.		
Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję *Wyświetl* tematu o statusie Odrzucony z listy tematów.		System wyświetla stronę szczegółów tematu (zgodnie z danymi z tabeli DT1), na której

		widoczne są: 1. Nazwa 2. Status (Odrzucony) 3. Opiekun (ciąg: [Tytuł] [Imię] [Nazwisko] lub znak „-” w przypadku braku). 4. Sekcja „Skład zespołu” ze studentami przypisanymi do tematu (każdy w osobnym wierszy: [Imię] [Nazwisko] lub znak „-” w przypadku braku) 5. Opis lub w przypadku braku „-”. 6. Przycisk z tekstem „Zobacz Opinię”.

1.5 Wystawianie opinii przez Członka KPK [Aleksandra Kozioł]

Członkowie KPK dostępni w systemie [DT2] – warunek wstępny.

Lp.	Imię	Nazwisko	Mail
1	Alicja	Kot	alicja.kot.1@pwr.edu.pl
2	Adam	Kot	adam.kot.2@pwr.edu.pl

Tematy

Lp.	**Nazwa**	**Opis**	**Status**	**Opiekun (tytuł + imię + nazwisko)**	**Studenci (imię + nazwisko)**	**Podpisy**
1	Modelowanie danych w NoSQL	Badanie wydajności oraz struktur przechowywania danych w bazach dokumentowych i grafowych w porównaniu do tradycyjnych relacyjnych baz danych.	Złożony	mgr Alicja Kozłowska	Krzysztof Kowalczyk, Magdalena Lewandowska Tomasz Kaczmarek, Natalia Mazur	mgr Alicja Kozłowska, Krzysztof Kowalczyk, Magdalena Lewandowska, Tomasz Kaczmarek, Natalia Mazur
2	Analiza wydajności aplikacji webowych	Porównanie wydajności aplikacji frontendowych opartych o różne frameworki JavaScript oraz techniki optymalizacji renderowania.	Złożony	prof. dr hab. inż. Natalia Michalska	Joanna Szymańska, Bartosz Woźniak, Alicja Dąbrowska	prof. dr hab. inż. Natalia Michalska, Joanna Szymańska, Bartosz Woźniak, Alicja Dąbrowska

3	Aplikacja mobilna wspierająca naukę	Stworzenie aplikacji mobilnej umożliwiającej planowanie nauki, śledzenie postępów oraz powtórki materiału.	Złożony	Piotr Jastrzębski	Łukasz Szybki, Ignacy Maj, Tomasz Nowy, Łukasz Szybki	Piotr Jastrzębski, Łukasz Szybki, Ignacy Maj, Tomasz Nowy, Łukasz Szybki

TC ID: 001
Idea testowa: Wystawienie pozytywnej opinii dla tematu
Warunki wstępne: W systemie znajduje się przynajmniej jeden temat, który posiada status Złożony oraz przynajmniej jeden Członek KPK (zgodnie z DT2). Użytkownik jest zalogowanym Członkiem KPK.

Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję „Zatwierdź” przy wybranym temacie.		System wyświetla modal z komunikatem „Zatwierdzono temat!” i przyciskiem „Zamknij”.
Użytkownik wybiera opcję „Zamknij”.		System przechodzi na stronę wyświetlania szczegółów tematu.

TC ID: 002
Idea testowa: Wystawienie negatywnej opinii dla tematu
Warunki wstępne: W systemie znajduje się przynajmniej jeden temat, który posiada status Złożony oraz przynajmniej jeden Członek KPK (zgodnie z DT2). Użytkownik jest zalogowanym Członkiem KPK.

Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję „Odrzuć” przy wybranym temacie.		System wyświetla widok odrzucenia tematu, zawierający (zgodnie z danymi z tabeli DT2): 1. Nazwę tematu 2. Opiekun tematu (ciąg: [Tytuł] [Imię] [Nazwisko] lub znak „-” w przypadku braku). 3. Decyzję (negatywna) 4. Pole tekstowe na uzasadnienie. 5. Przycisk „Dodaj”
Użytkownik wprowadza uzasadnienie i klika „Dodaj”.	**Lp.** / **Uzasadnienie** 1 / Zbyt mało szczegółowo opisany temat. 2 / ?ąę12 []/9& 3\’ 3 / [Tekst o długości 1000 znaków]	System wyświetla modal z komunikatem „Odrzucono Temat!” i przyciskiem „Zamknij”.
Użytkownik wybiera opcję „Zamknij”.		System przechodzi na stronę wyświetlania szczegółów tematu.

TC ID: 003
Idea testowa: Weryfikacja walidacji pola uzasadnienia przy wystawianiu negatywnej opinii
Warunki wstępne: W systemie znajduje się przynajmniej jeden temat, który posiada status Złożony oraz przynajmniej jeden Członek KPK (zgodnie z DT2). Użytkownik jest zalogowanym Członkiem KPK.

Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję „Odrzuć” przy wybranym temacie.		System wyświetla widok odrzucenia tematu, zawierający (zgodnie z danymi z tabeli DT2): 1. Nazwę tematu 2. Opiekuna tematu (ciąg: [Tytuł] [Imię] [Nazwisko] lub znak „-” w przypadku braku). 3. Decyzję (negatywna) 4. Pole tekstowe na uzasadnienie. 5. Przycisk „Dodaj”
Użytkownik pozostawia pole puste lub wpisuje same białe znaki i klika „Dodaj”.	Lp. / Uzasadnienie 1 / „” 2 / „ ”	System wyświetla modal z komunikatem „Nie podano uzasadnienia!” i przyciskiem „Zamknij”.
Użytkownik wybiera opcję „Zamknij”.		System przechodzi na stronę wyświetlania szczegółów tematu.

TC ID: 004 Idea testowa: Wyście podczas wystawiania negatywnej opinii gdy nie wprowadzono uzasadnienia Warunki wstępne: W systemie znajduje się przynajmniej jeden temat, który posiada status Złożony oraz przynajmniej jeden Członek KPK (zgodnie z DT2). Użytkownik jest zalogowanym Członkiem KPK.		
Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję „Odrzuć” przy wybranym temacie.		System wyświetla widok odrzucenia tematu, zawierający (zgodnie z danymi z tabeli DT2): 1. Nazwę tematu 2. Opiekun tematu (ciąg: [Tytuł] [Imię] [Nazwisko] lub znak „-” w przypadku braku). 3. Decyzję (negatywna) 4. Pole tekstowe na uzasadnienie. 5. Przycisk „Dodaj”
Użytkownik pozostawia pole puste lub wpisuje same białe znaki, a następnie wybiera opcję „Wróć” aby wyjść z widoku Odrzucenia tematu.	Lp. / Uzasadnienie 1 / „” 2 / „ ”	System zamyka widok odrzucenia, a wyświetla widok szczegółów tematu.

TC ID: 005 Idea testowa: Potwierdzenie wyjścia z widoku odrzucenia (porzucenie zmian) Warunki wstępne: W systemie znajduje się przynajmniej jeden temat, który posiada status Złożony oraz przynajmniej jeden Członek KPK (zgodnie z DT2). Użytkownik jest zalogowanym Członkiem KPK.		
Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję „Odrzuć” przy wybranym temacie.		System wyświetla widok odrzucenia tematu, zawierający (zgodnie z danymi z tabeli DT2): 1. Nazwę tematu 2. Opiekun tematu (ciąg: [Tytuł] [Imię] [Nazwisko] lub znak „-” w przypadku braku).

		3. Decyzję (negatywna) 4. Pole tekstowe na uzasadnienie. 5. Przycisk „Dodaj”
Użytkownik wprowadza uzasadnienie i wybiera opcję „Wróć” aby wyjść z widoku Odrzucenia tematu.	Lp. / Uzasadnienie 1 / Zbyt mało szczegółowo opisany temat. 2 / ?ąę12 []/9& 3\’ 3 / [Tekst o długości 1000 znaków]	System wyświetla modal ostrzegawczy z komunikatem "Masz niezapisane zmiany. Czy na pewno chcesz opuścić stronę?" oraz przyciskami „Anuluj”, „Przerwij”.
Użytkownik klika przycisk „Przerwij”		Modal oraz widok odrzucenia zostają zamknięte. System wyświetla widok szczegółów tematu.

TC ID: 006
Idea testowa: Anulowanie wyjścia z widoku odrzucenia (powrót do edycji).
Warunki wstępne: W systemie znajduje się przynajmniej jeden temat, który posiada status Złożony oraz przynajmniej jeden Członek KPK (zgodnie z DT2). Użytkownik jest zalogowanym Członkiem KPK.

Krok testu	Dane testowe	Oczekiwane wyniki
Użytkownik wybiera opcję „Odrzuć” przy wybranym temacie.		System wyświetla widok odrzucenia tematu, zawierający (zgodnie z danymi z tabeli DT2): 1. Nazwę tematu 2. Opiekun tematu (ciąg: [Tytuł] [Imię] [Nazwisko] lub znak „-” w przypadku braku). 3. Decyzję (negatywna) 4. Pole tekstowe na uzasadnienie. 5. Przycisk „Dodaj”
Użytkownik wprowadza uzasadnienie i wybiera opcję „Wróć” aby wyjść z widoku Odrzucenia tematu.	Lp. / Uzasadnienie 1 / Zbyt mało szczegółowo opisany temat. 2 / ?ąę12 []/9& 3\’ 3 / [Tekst o długości 1000 znaków]	System wyświetla modal ostrzegawczy z komunikatem "Masz niezapisane zmiany. Czy na pewno chcesz opuścić stronę?" oraz przyciskami „Anuluj”, „Przerwij”.
Użytkownik klika przycisk „Anuluj”		Modal znika. Użytkownik pozostaje w widoku odrzucenia. Wprowadzona treść uzasadnienia nadal jest widoczna w polu tekstowym.

2. Testy systemowe automatyczne

Do automatycznych testów funkcjonalnych wykorzystano narzędzie Cypress. Ten framework pozwala na wykonanie automatycznych testów z perspektywy użytkownika końcowego, które pozwalają między innymi na przechodzenie do danej ścieżki, sprawdzenie, czy ścieżka jest odpowiednia, czy wymagane elementy znajdują się, są widoczne na stronie, pozwala na symulowanie działań użytkownika, takich jak klikanie przycisków, czy wypełnianie pół tekstowych. Podjęto się automatyzacji testów dla przeglądania tematów i ich szczegółów, dodawania opinii, dopisywania i wypisywania się z tematu oraz podpisywania deklaracji przez studenta. Przykładowy test – wypisanie się z tematu:

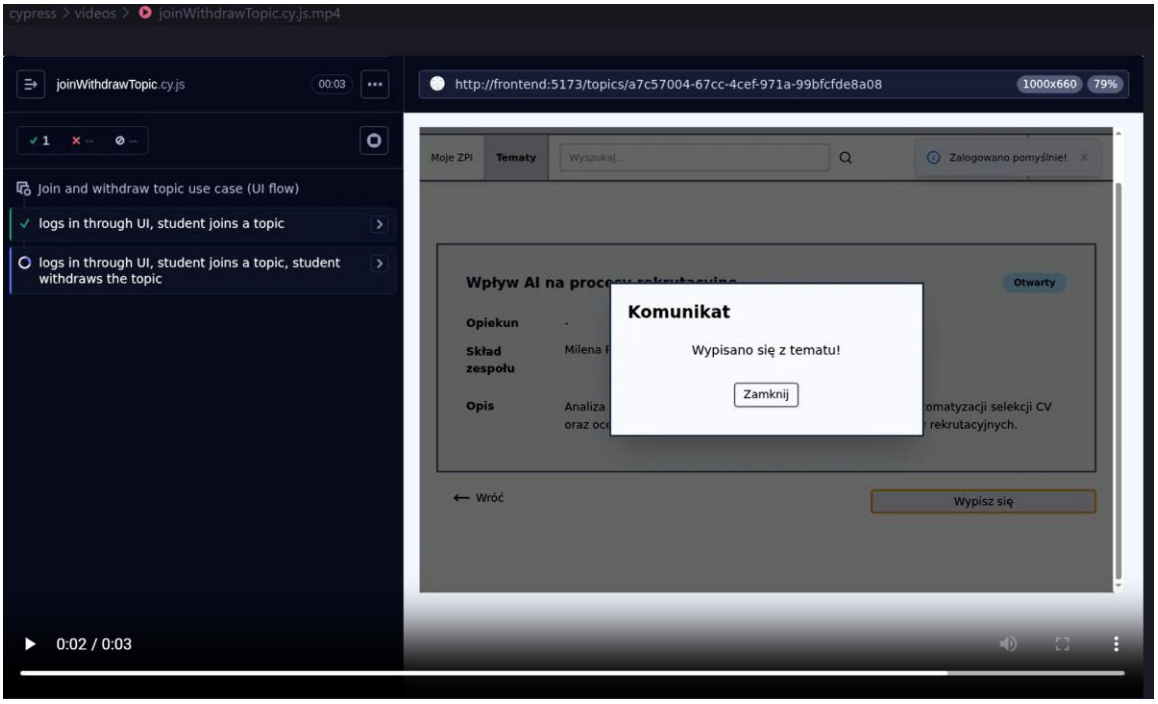
```
3   describe('Join and withdraw topic use case (UI flow)', () => {
34
35    it('logs in through UI, student joins a topic, student withdraws the topic', () => {
36      cy.visit('/');
37
38      cy.contains(STUDENT_EMAIL).click();
39      cy.contains('button', /^Zaloguj/i).click();
40
41      cy.url().should('include', '/topics');
42
43      cy.contains(TOPIC_NAME)
44        .closest('tr')
45        .within(() => {
46          cy.contains('Wyświetl').click();
47        });
48
49      cy.contains('span', STATUSES.OPEN).should('be.visible');
50
51      cy.contains('button', 'Wypisz się').click();
52
53      cy.contains('Wypisano się z tematu!').should('be.visible');
54
55      cy.contains('button', 'Zamknij').click();
56
57      cy.get(`[data-testid="people-table"]`).should('not.contain', STUDENT_NAME)
58
59      cy.contains('Wyloguj się').click();
60    });
61  });
62
```

Jest to fragment pliku *joinWithdrawTopic.cy.js*. Wszystkie testy funkcjonalne znajdują się w folderze *cypress/e2e*. Na górze pliku zdefiniowano dane studenta i tematu, dla którego będzie on przeprowadzany i zapewniono poprawność warunków początkowych. Każdy z testów jest niezależny od siebie, a *Cypress* uruchamia je bez definiowania kolejności. Poza tym zapewniono, aby przed wykonaniem testów, wygenerować odpowiedni stan bazy danych, a po testach przywrócić ją do stanu początkowego.

Wyniki testów można sprawdzić w konsoli. Fragment wyniku z przeprowadzonego testu:

```
ZPI-cypress  |    ✓ logs in through UI, student joins a topic, student withdraws the topic (1250ms)
ZPI-cypress  |
ZPI-cypress  |
ZPI-cypress  |  2 passing (3s)
ZPI-cypress  |
ZPI-cypress  |
ZPI-cypress  |  (Results)
ZPI-cypress  |
ZPI-cypress  |  ┌──────────────────────────────────────────────────────────────────────────────────────────────┐
ZPI-cypress  |  │ Tests:        2                                                                              │
ZPI-cypress  |  │ Passing:      2                                                                              │
ZPI-cypress  |  │ Failing:      0                                                                              │
ZPI-cypress  |  │ Pending:      0                                                                              │
ZPI-cypress  |  │ Skipped:      0                                                                              │
ZPI-cypress  |  │ Screenshots:  0                                                                              │
ZPI-cypress  |  │ Video:        true                                                                           │
ZPI-cypress  |  │ Duration:     3 seconds                                                                      │
ZPI-cypress  |  │ Spec Ran:     joinWithdrawTopic.cy.js                                                        │
ZPI-cypress  |  └──────────────────────────────────────────────────────────────────────────────────────────────┘
```

Oprócz wypisania wyników testów w formie tekstowej, jedną z funkcjonalności *Cypressa* jest automatyczne generowanie filmów z przebiegu testu, co pozwala prześledzić wykonywane działania oraz ewentualne błędy.



3. Testy jednostkowe

Testy jednostkowe stworzono dla funkcji z folderu *services,* odpowiadających za zaimplementowane przypadki użycia, tj. funkcji z plików *assignmentService.js (joinTopic, withdrawTopic), declarationService.js (signDeclaration), opinionService.js (addOpinion), topicsService.js (mapTopicToDto, getAllTopics, getTopicByUuid, updateStatus).* Pliki z testami znajdują się w podkatalogu *__tests__*. Jako że w projekcie wykorzystano narzędzie *Vite*, do testów jednostkowych postanowiono użyć, opartego na *Vite*¸ frameworka *Vitest*.

Vitest pozwala na zamockowanie komunikacji, zapytań i odpowiednich odpowiedzi, z bazą danych. W ramach testu wykonywane są wszystkie niezbędne operacji, a następnie weryfikowany jest wynik wykonanej funkcji w danym przypadku. Przykładowo poniżej znajduje się fragment kodu dla testu dodawania opinii, który weryfikuje poprawne utworzenie pozytywnej opinii i zmianę statusu.

```
82    describe("Positive Opinion", () => {
83      it("creates opinion and updates status to APPROVED when isPositive true", async () => {
84        prismaMock.topic.findUnique.mockResolvedValue({
85          topic_id: 7,
86          status: { status_name: STATUSES.SUBMITTED }
87        });
88        prismaMock.opinion.create.mockResolvedValue({});
89        findStatus.mockResolvedValue({ status_id: 123 });
90
91        await addOpinion("t-uuid", "good", true, 11);
92
93        expect(prismaMock.opinion.create).toHaveBeenCalledWith({
94          data: { argumentation: "good", employee_id: 11, topic_id: 7 }
95        });
96        expect(findStatus).toHaveBeenCalledWith(STATUSES.APPROVED);
97        expect(updateStatus).toHaveBeenCalledWith("t-uuid", 123);
98      });
99
```

4. Ocena jakości kodu

Do oceny jakości kodu użyto narzędzia Qlty CLI. To narzędzie oferuje trzy główne komendy, których wyniki opisano poniżej. Pełne wyniki znajdują się w folderze /metrics projektu, podfolder /initial zawiera początkowe wyniki, a podfolder /fixed zawiera metryki po poprawkach. Wyniki, w zależności od ich formatu, zapisano jako pliki txt, json oraz html.

Pierwsza komenda to *check*, która wypisuje liczbę znalezionych problemów wraz z krótkim opisem.

```
ISSUES: 12


backend/prisma/seeder.js:402:402
402:402  medium  Remove this useless assignment to variable "teachingSupervisors".     radarlint-js:javascript:S1854
402:402  low     Remove the declaration of the unused 'teachingSupervisors' variable.  radarlint-js:javascript:S1481
403:403  medium  Remove this useless assignment to variable "studentsSupervisors".     radarlint-js:javascript:S1854
403:403  low     Remove the declaration of the unused 'studentsSupervisors' variable.  radarlint-js:javascript:S1481

backend/src/middleware/auth.js:8:8
    8:8  low     Prefer using an optional chain expression instead, as it's more concise and easier to read.  radarlint-js:javascript:S6582

backend/src/services/__tests__/assignmentsService.test.js:15:15
  15:15  low     Remove this unused import of 'MAX_TOPIC_CAPACITY'.  radarlint-js:javascript:S1128

backend/src/services/__tests__/topicsService.test.js:12:12
  12:12  low     Remove this unused import of 'updateStatus'.  radarlint-js:javascript:S1128

backend/src/services/opinionsService.js:44:44
  44:44  low     Refactor the code to avoid using this boolean literal.  radarlint-js:javascript:S1125

backend/src/services/signaturesService.js:56:56
  56:56  low     Prefer using an optional chain expression instead, as it's more concise and easier to read.  radarlint-js:javascript:S6582

frontend/src/api/apiFetch.js:16:16
  16:16  medium  Default parameters should be last.  radarlint-js:javascript:S1788

frontend/src/hooks/useTopicsLogic.js:71:71
  71:71  high    Refactor this function to reduce its Cognitive Complexity from 20 to the 15 allowed.  radarlint-js:javascript:S3776
105:105  high    Refactor this function to reduce its Cognitive Complexity from 19 to the 15 allowed.  radarlint-js:javascript:S3776
```

Głównie zidentyfikowane problemy oznaczone jako *low*, które były bardzo proste do poprawy typu nieużywane importy, zmienne które nie były używane czy możliwość skorzystania z operatora ? zamiast sprawdzania czy zmienna jest undefined. Był jeden błąd *medium* spowodowany tym, że parametr z wartością domyślną nie był na ostatnim miejscu, co też zmieniłyśmy Najważniejszym wyzwaniem były jednak dwa błędy high dotyczące zbyt dużej Cognitive Complexity w pliku useTopicsLogic.js, co jasno

wskazywało, że kod jest zbyt złożony i będzie ciężki do utrzymania. W celu naprawy dokonałyśmy dekompozycję tej metody, w której wcześniej była ubita cała logika filtrowania i sortowania. Rozbiłyśmy ją na małe, atomowe funkcje pomocnicze, takie jak matchesStatus czy matchesSupervisor, i wyciągnęłyśmy logikę wyciągania danych do obiektu GET_VALUE. Dzięki temu kod jest teraz czytelniejszy i łatwiejszy w utrzymaniu.

Po poprawkach, komenda *check* nie wykryła żadnych błędów.

Drugą wykorzystaną komendą było *smells,* która znajduje potencjalne problemy w kodzie, które nie muszą być błędami, ale które mogą prowadzić do późniejszych problemów. W przypadku naszego kodu wykryto funkcje o wielu zwracanych wartościach (wiele *return*) oraz przykład podobnego kodu.

Fragment kodu znajdujący się w pliku frontend/src/hooks/useAuth.js został rozpoznany jako kod przypominający fragment z pliku useLogin.js.

```
frontend/src/hooks/useAuth.js
    Found 55 lines of similar code in 2 locations (mass = 252)

         111  export const useLogin = () => {
         112    const { login: setAuthContext } = useAuthContext();
         113    const [users, setUsers] = useState([]);
         114    const [loadingUsers, setLoadingUsers] = useState(true);
         115    const [loggingIn, setLoggingIn] = useState(false);
         116    const [error, setError] = useState("");
              [hid 49 additional lines]
        also found at frontend/src/hooks/useLogin.js
```

Zanalizowano oba pliki i stwierdzono, że kod zawarty w pliku useLogin.js nie jest wykorzystywany przez aplikację. Poprawiono kod tak, że cała funkcjonalność jest obecnie realizowana przez useAuth.js, a plik useLogin.js usunięto.

W pliku frontend/src/hooks/useTopicActionsHandlers.js odkryto problem wielu returnów, m.in. związnymi z isActionAllowed, które służy do sprawdzenia, czy użytkownik może wykonać daną akcję, np. w zależności od jego roli.

```
frontend/src/hooks/useTopicActionsHandlers.js
    Function with many returns (count = 12): useTopicActions

          78  export const useTopicActions = (topic, signatures, isAssignedToAnyTopic) => {
          79    const { user } = useAuthContext();
          80    const handlers = useTopicActionsHandlers();
          81
          82    const allowedActions = useMemo(() => {
          83      if (!topic || !user) return [];
              [hid 42 additional lines]

    Function with many returns (count = 9): isActionAllowed

          93      const isActionAllowed = (actionId) => {
          94        switch (actionId) {
          95          case TOPIC_ACTIONS.JOIN:
          96            return !isStudentTeamMember && !isAssignedToAnyTopic;
          97          case TOPIC_ACTIONS.WITHDRAW:
          98            return isStudentTeamMember;
              [hid 16 additional lines]
```

Instrukcję switch-case zastąpiono actionPermissions, niwelując ten problem.

```
const actionPermissions = {
  [TOPIC_ACTIONS.JOIN]: () => !isStudentTeamMember,
  [TOPIC_ACTIONS.WITHDRAW]: () => isStudentTeamMember,
  [TOPIC_ACTIONS.SIGN]: () => isUserTeamMember && !hasSigned,
  [TOPIC_ACTIONS.VIEW_OPINION]: () => user.role === ROLES.KPK_MEMBER || isUserTeamMember,
  [TOPIC_ACTIONS.ADD_SUPERVISION]: () => !hasSupervisor,
  [TOPIC_ACTIONS.DEL_SUPERVISION]: () => hasSupervisor && isUserSupervisor,
  [TOPIC_ACTIONS.ADD_STUDENT]: () => isUserSupervisor || user.role === ROLES.STUDENTS_SUPERVISOR,
  [TOPIC_ACTIONS.DEL_STUDENT]: () =>
    (isUserSupervisor || user.role === ROLES.STUDENTS_SUPERVISOR) &&
    topic.students?.length > 0,
};

const isActionAllowed = (actionId) =>
  actionPermissions[actionId]?.() ?? true;
```

W podobny sposób rozwiązano analogiczny problem zlokalizowany w pliku frontend/src/components/NotificationContainer.jsx.

Kodu z backend/src/middleware/errorHandler.js, mimo wykrycia dużej liczby instrukcji return, zdecydowano się nie zmieniać. Kod *errorHandler* służy jako kod pomocniczy dla zdefiniowanych specjalnych wyjątków, zapewnia wygodę rzucania różnych wyjątków w innych funkcjach aplikacji. Uznano, że poprawa nie jest konieczna.

Po wprowadzeniu zmian do kodu, w wyniku komendy *smells* pozostała jedynie uwaga odnoście *errorHandler.*

Ostatnią komendą użytą do analizy jakości kodu było *metrics*, która wyświetla wyniki poszczególnych metryk z podziałem na pliki oraz wartości sumaryczne. Pełne wyniki komendy zamieszczono poniżej.

name	classes	funcs	fields	cyclo	complex	LCOM	lines	LOC
backend/prisma.config.ts	0	0	0	1	0	0	14	11
backend/prisma/seeder.js	0	6	0	21	13	0	465	408
backend/src/controllers/academicEmployeesController.js	0	2	0	1	0	0	32	13
backend/src/controllers/assignmentController.js	0	2	0	1	0	0	42	16
backend/src/controllers/authController.js	0	3	0	7	2	0	83	42
backend/src/controllers/declarationController.js	0	1	0	1	0	0	22	9
backend/src/controllers/opinionsController.js	0	1	0	1	0	0	25	10
backend/src/controllers/signaturesController.js	0	1	0	1	0	0	17	6
backend/src/controllers/studentsController.js	0	3	0	1	0	0	49	19
backend/src/controllers/topicsController.js	0	2	0	1	0	0	36	14
backend/src/controllers/usersController.js	0	1	0	1	0	0	15	5
backend/src/lib/db.js	0	1	0	4	1	0	20	13
backend/src/middleware/auth.js	0	2	0	7	6	0	34	26
backend/src/middleware/errorHandler.js	0	1	0	28	10	0	86	72
backend/src/routes/academicEmployees.js	0	0	0	1	0	0	9	6
backend/src/routes/auth.js	0	0	0	1	0	0	11	8
backend/src/routes/opinions.js	0	0	0	1	0	0	9	6
backend/src/routes/signatures.js	0	0	0	1	0	0	8	6
backend/src/routes/students.js	0	0	0	1	0	0	10	7
backend/src/routes/topics.js	0	0	0	1	0	0	24	20
backend/src/routes/user.js	0	0	0	1	0	0	8	5
backend/src/server.js	0	0	0	2	0	0	48	39
backend/src/services/academicEmployeesService.js	0	3	0	3	1	0	87	67
backend/src/services/assignmentsService.js	0	4	0	14	7	0	107	67
backend/src/services/authService.js	0	4	0	4	3	0	79	33
backend/src/services/declarationService.js	0	2	0	15	8	0	78	50
backend/src/services/opinionsService.js	0	1	0	7	4	0	49	29
backend/src/services/signaturesService.js	0	2	0	4	2	0	61	43
backend/src/services/statusService.js	0	1	0	1	0	0	13	4
backend/src/services/studentsService.js	0	4	0	4	1	0	89	61
backend/src/services/topicsService.js	0	4	0	7	4	0	105	67
backend/src/services/usersService.js	0	6	0	3	1	0	92	55
backend/src/utils/errors.js	7	7	5	1	0	0	48	40
frontend/src/App.jsx	0	3	0	2	1	0	123	108
frontend/src/api/apiFetch.js	0	2	0	13	7	0	51	42
frontend/src/components/AnnouncementModal.jsx	0	5	0	14	9	0	75	65
frontend/src/components/BackButton.jsx	0	2	0	1	0	0	22	18
frontend/src/components/Navbar.jsx	0	9	0	15	12	0	123	102
frontend/src/components/NotificationContainer.jsx	0	5	0	14	3	0	68	58
frontend/src/components/ProtectedRoute.jsx	0	1	0	7	4	0	26	23
frontend/src/components/TopicActionButtons.jsx	0	1	0	4	1	0	23	20
frontend/src/components/TopicsFIlterPanel.jsx	0	8	0	23	8	0	111	98
frontend/src/components/TopicsTable.jsx	0	3	0	13	7	0	83	78
frontend/src/contexts/AuthContext.js	0	1	0	2	1	0	11	9
frontend/src/contexts/ModalContext.js	0	1	0	2	1	0	11	9
frontend/src/contexts/NotificationContext.js	0	1	0	2	1	0	11	9
frontend/src/hooks/useActionRequest.js	0	2	0	5	3	0	63	49
frontend/src/hooks/useApi.js	0	1	0	3	2	0	36	23
frontend/src/hooks/useAuth.js	0	6	0	8	9	0	165	115
frontend/src/hooks/useLogin.js	0	3	0	3	2	0	59	51
frontend/src/hooks/useTopicActionsHandlers.js	0	5	0	31	8	0	125	93
frontend/src/hooks/useTopicsLogic.js	0	3	0	56	52	0	148	106
frontend/src/hooks/useUnsavedChanges.jsx	0	4	0	6	3	0	50	43
frontend/src/main.jsx	0	0	0	1	0	0	7	6
frontend/src/pages/LoginPage.jsx	0	3	0	12	6	0	86	72
frontend/src/pages/OpinionFormPage.jsx	0	6	0	16	7	0	179	148
frontend/src/pages/TopicPage.jsx	0	3	0	30	15	0	200	171
frontend/src/pages/TopicsPage.jsx	0	4	0	14	8	0	112	97
frontend/src/pages/UserPage.jsx	0	1	0	1	0	0	37	34
frontend/src/pages/ZPIPage.jsx	0	1	0	1	0	0	47	39
frontend/src/providers/AuthProvider.jsx	0	1	0	1	0	0	12	10
frontend/src/providers/ModalProvider.jsx	0	1	0	1	0	0	30	24
frontend/src/providers/NotificationProvider.jsx	0	1	0	6	1	0	46	37
TOTAL	7	151	5	455	234	0	4015	3034

Jako że kod jest napisany w JavaScripcie, nie występiło w nim wiele klas, za wyjątkiem pliku backend/src/utils/errors.js, w którym zdefiniowano konkretne wyjątki, rzucane przez naszą aplikację. Liczba funkcji w pojedynczym pliku nie przekracza 9, przy czym średnio w jednym pliku znajdują się 2-3, co uznano za dobry wynik. Jednocześnie pliki nie przekraczają 175 linii kodu, za wyjątkiem pliku backend/prisma/seeder.js (408 linii kodu), gdzie zamieszczono funkcje do wypełnienia bazy danych przykładowymi wartościami.
Zanalizowano także wartości metryk *cyclo* (Cyclomatic complexity) oraz *complex* (Cognitive Complexity). Około 75% plików nie przekracza wartości 10 złożoności cyklometrycznej, co oznacza że większość kodu jest odpowiednio mało złożona. Około 60% z pozostałych plików, ma wartość poniżej 20, a więc są to pliki

średnio złożone. Pliki o wysokiej złożoności to

- backend/src/controllers/academicEmployeesController.js
- backend/src/middleware/errorHandler.js
- frontend/src/components/TopicsFIlterPanel.jsx
- frontend/src/hooks/useTopicActionsHandlers.js
- frontend/src/hooks/useTopicsLogic.js
- frontend/src/pages/TopicPage.jsx

Przy czym na szczególną uwagę zasługuje plik useTopicsLogic.js, którego złożoność cyklometryczna przekroczyła wartość 50. W przypadku metryki *complex,* dla prawie wszystkich plików wartości nie przekraczają 15, co świadczy o dobrej czytelności kodu. Wyjątkiem jest jedynie wcześniej wspomniany plik useTopicsLogic.js, dla którego wartość metryki ponownie przekroczyła 50.

Po wprowadzeniu poprawek w kodzie, ponownie użyto komendy *metrics.* Mimo że dla pliku useTopicsLogic.js wartości złożoności nadal są stosunkowo wysokie, to zauważono poprawę w porównaniu do poprzednich wyników metryk, zwłaszcza w przypadku metyki *complex*. Udało się także zmniejszyć wartość złożoności dla kodu z pliku useTopicActionsHandlers.js oraz TopicPage.jsx. Pełne wyniki znajdują się na następnej stronie.

name	classes	funcs	fields	cyclo	complex	LCOM	lines	LOC
backend/prisma.config.ts	0	0	0	1	0	0	14	11
backend/prisma/seeder.js	0	6	0	21	13	0	465	408
backend/src/controllers/academicEmployeesController.js	0	2	0	1	0	0	32	13
backend/src/controllers/assignmentController.js	0	2	0	1	0	0	42	16
backend/src/controllers/authController.js	0	3	0	7	2	0	83	42
backend/src/controllers/declarationController.js	0	1	0	1	0	0	22	9
backend/src/controllers/opinionsController.js	0	1	0	1	0	0	25	10
backend/src/controllers/signaturesController.js	0	1	0	1	0	0	17	6
backend/src/controllers/studentsController.js	0	3	0	1	0	0	49	19
backend/src/controllers/topicsController.js	0	2	0	1	0	0	36	14
backend/src/controllers/usersController.js	0	1	0	1	0	0	15	5
backend/src/lib/db.js	0	1	0	4	1	0	20	13
backend/src/middleware/auth.js	0	2	0	6	5	0	34	26
backend/src/middleware/errorHandler.js	0	1	0	28	10	0	86	72
backend/src/routes/academicEmployees.js	0	0	0	1	0	0	9	6
backend/src/routes/auth.js	0	0	0	1	0	0	11	8
backend/src/routes/opinions.js	0	0	0	1	0	0	9	6
backend/src/routes/signatures.js	0	0	0	1	0	0	8	6
backend/src/routes/students.js	0	0	0	1	0	0	10	7
backend/src/routes/topics.js	0	0	0	1	0	0	24	20
backend/src/routes/user.js	0	0	0	1	0	0	8	5
backend/src/server.js	0	0	0	2	0	0	48	39
backend/src/services/academicEmployeesService.js	0	3	0	3	1	0	87	67
backend/src/services/assignmentsService.js	0	4	0	14	7	0	107	67
backend/src/services/authService.js	0	4	0	4	3	0	79	33
backend/src/services/declarationService.js	0	2	0	15	8	0	78	50
backend/src/services/opinionsService.js	0	1	0	7	4	0	49	29
backend/src/services/signaturesService.js	0	2	0	3	1	0	61	43
backend/src/services/statusService.js	0	1	0	1	0	0	13	4
backend/src/services/studentsService.js	0	4	0	4	1	0	89	61
backend/src/services/topicsService.js	0	4	0	7	4	0	105	67
backend/src/services/usersService.js	0	6	0	3	1	0	92	55
backend/src/utils/errors.js	7	7	5	1	0	0	48	40
cypress.config.js	0	0	0	2	1	0	8	8
frontend/eslint.config.js	0	0	0	1	0	0	29	28
frontend/src/App.jsx	0	3	0	2	1	0	123	108
frontend/src/api/apiFetch.js	0	2	0	13	7	0	51	42
frontend/src/components/AnnouncementModal.jsx	0	5	0	14	9	0	75	65
frontend/src/components/BackButton.jsx	0	2	0	1	0	0	22	18
frontend/src/components/Navbar.jsx	0	9	0	15	12	0	123	102
frontend/src/components/NotificationContainer.jsx	0	5	0	6	1	0	65	56
frontend/src/components/ProtectedRoute.jsx	0	1	0	7	4	0	26	23
frontend/src/components/TopicActionButtons.jsx	0	1	0	4	1	0	23	20
frontend/src/components/TopicsFIlterPanel.jsx	0	8	0	23	8	0	111	98
frontend/src/components/TopicsTable.jsx	0	3	0	13	7	0	88	83
frontend/src/contexts/AuthContext.js	0	1	0	2	1	0	11	9
frontend/src/contexts/ModalContext.js	0	1	0	2	1	0	11	9
frontend/src/contexts/NotificationContext.js	0	1	0	2	1	0	11	9
frontend/src/hooks/useActionRequest.js	0	2	0	5	3	0	63	49
frontend/src/hooks/useApi.js	0	1	0	3	2	0	36	23
frontend/src/hooks/useAuth.js	0	6	0	8	9	0	164	114
frontend/src/hooks/useTopicActionsHandlers.js	0	5	0	22	7	0	118	86
frontend/src/hooks/useTopicsLogic.js	0	14	0	40	15	0	222	84
frontend/src/hooks/useUnsavedChanges.jsx	0	4	0	6	3	0	50	43
frontend/src/main.jsx	0	0	0	1	0	0	7	6
frontend/src/pages/LoginPage.jsx	0	3	0	12	6	0	86	72
frontend/src/pages/OpinionFormPage.jsx	0	6	0	17	9	0	179	148
frontend/src/pages/TopicPage.jsx	0	2	0	26	12	0	165	143
frontend/src/pages/TopicsPage.jsx	0	4	0	14	8	0	112	97
frontend/src/pages/UserPage.jsx	0	1	0	1	0	0	37	34
frontend/src/pages/ZPIPage.jsx	0	1	0	1	0	0	47	39
frontend/src/providers/AuthProvider.jsx	0	1	0	1	0	0	12	10
frontend/src/providers/ModalProvider.jsx	0	1	0	1	0	0	30	24
frontend/src/providers/NotificationProvider.jsx	0	1	0	6	1	0	46	37
frontend/vite.config.js	0	0	0	1	0	0	24	22
TOTAL	7	158	5	418	190	0	4050	2986